

# Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Documentation technique – <code>nbbbenuts</code></b>                | <b>2</b> |
| 1.1      | Table des matieres . . . . .                                           | 2        |
| 1.2      | 1. Vue d’ensemble . . . . .                                            | 3        |
| 1.3      | 2. Les 23 classifications supportees . . . . .                         | 3        |
| 1.4      | 3. Le graphe de conversion . . . . .                                   | 4        |
| 1.4.1    | 3.1 Aretes primitives . . . . .                                        | 4        |
| 1.4.2    | 3.2 Aretes inverses . . . . .                                          | 4        |
| 1.4.3    | 3.3 Recherche de chemin (BFS) . . . . .                                | 5        |
| 1.4.4    | 3.4 Conversion multi-saut . . . . .                                    | 5        |
| 1.4.5    | 3.5 Conversions notables . . . . .                                     | 5        |
| 1.5      | 4. Semantique de perimetre . . . . .                                   | 5        |
| 1.6      | 5. La colonne <b>nature</b> . . . . .                                  | 6        |
| 1.7      | 6. API complete . . . . .                                              | 7        |
| 1.7.1    | 6.1 Chargement des donnees . . . . .                                   | 7        |
| 1.7.2    | 6.2 Conversion de codes . . . . .                                      | 7        |
| 1.7.3    | 6.3 Conversion d’un dataset . . . . .                                  | 8        |
| 1.7.4    | 6.4 Inspection des chemins . . . . .                                   | 8        |
| 1.7.5    | 6.5 Validation et labels . . . . .                                     | 9        |
| 1.7.6    | 6.6 Tables de correspondance . . . . .                                 | 9        |
| 1.7.7    | 6.7 Detection et diagnostic . . . . .                                  | 10       |
| 1.7.8    | 6.8 Poids et desagregation . . . . .                                   | 10       |
| 1.7.9    | 6.9 Rebasage longitudinal . . . . .                                    | 11       |
| 1.7.10   | 6.10 Correspondance floue . . . . .                                    | 11       |
| 1.7.11   | 6.11 Objet <b>nomenclature</b> . . . . .                               | 12       |
| 1.7.12   | 6.12 Visualisation . . . . .                                           | 12       |
| 1.8      | 7. Architecture interne . . . . .                                      | 13       |
| 1.8.1    | 7.1 Flot d’une conversion . . . . .                                    | 13       |
| 1.8.2    | 7.2 <b>CLASSIFICATION_NODES</b> . . . . .                              | 13       |
| 1.8.3    | 7.3 Moteur de conversion (Phase 2) . . . . .                           | 14       |
| 1.8.4    | 7.4 Normalisation du resultat . . . . .                                | 14       |
| 1.9      | 8. Modele de donnees <b>master_data</b> . . . . .                      | 14       |
| 1.9.1    | <b>communes</b> – table large . . . . .                                | 14       |
| 1.9.2    | <b>postal</b> – codes postaux . . . . .                                | 15       |
| 1.9.3    | <b>nis_changes</b> – transitions de version . . . . .                  | 15       |
| 1.9.4    | <b>entities</b> – enumeration des membres par classification . . . . . | 15       |

|        |                                                                     |    |
|--------|---------------------------------------------------------------------|----|
| 1.9.5  | <b>crosswalks</b> – liens primitifs entre classifications . . . . . | 15 |
| 1.10   | 9. Classes d’erreur et avertissements . . . . .                     | 15 |
| 1.11   | 10. Suite de tests . . . . .                                        | 16 |
| 1.12   | 11. Reconstruction du snapshot . . . . .                            | 17 |
| 1.12.1 | Fichiers bruts requis ( <b>data/raw/</b> ) . . . . .                | 17 |
| 1.12.2 | Commandes . . . . .                                                 | 17 |
| 1.12.3 | Schema de validation . . . . .                                      | 17 |
| 1.13   | 12. Datasets d’exemple . . . . .                                    | 17 |
| 1.13.1 | Datasets propres (couverture complete) . . . . .                    | 17 |
| 1.13.2 | Dataset sale ( <b>rc_dirty_municipalities_2019</b> ) . . . . .      | 18 |
| 1.13.3 | Regeneration . . . . .                                              | 18 |
| 1.14   | Glossaire . . . . .                                                 | 18 |

## 1 Documentation technique – **nbbbenuts**

Reference technique complete du package R de conversion de codes geographiques belges. Pour les instructions d’utilisation rapide, voir **vignettes/introduction.Rmd**. Pour l’historique du refactoring, voir **REFACTORING\_PLAN.md**.

---

### 1.1 Table des matieres

1. Vue d’ensemble
2. Les 23 classifications supportees
3. Le graphe de conversion
4. Semantique de perimetre
5. La colonne **nature**
6. API complete
  - 6.1 Chargement des donnees
  - 6.2 Conversion de codes
  - 6.3 Conversion d’un dataset
  - 6.4 Inspection des chemins
  - 6.5 Validation et labels
  - 6.6 Tables de correspondance
  - 6.7 Detection et diagnostic
  - 6.8 Poids et desagregation
  - 6.9 Rebasage longitudinal
  - 6.10 Correspondance floue
  - 6.11 Objet **nomenclature**
  - 6.12 Visualisation
7. Architecture interne
8. Modele de donnees **master\_data**
9. Classes d’erreur et avertissements
10. Suite de tests

11. Reconstruction du snapshot
12. Datasets d'exemple

## 1.2 1. Vue d'ensemble

`nbbbenuts` convertit des **codes géographiques belges** entre systèmes de classification et à travers le temps. Il couvre quatre systèmes :

| Systeme                | Identifiant                                                                 | Exemples de codes                                  |
|------------------------|-----------------------------------------------------------------------------|----------------------------------------------------|
| <b>NIS</b> (Statbel)   | <code>NIS_*_{BEFORE_2019,2019,2025}</code>                                  | 21004 (Bruxelles), 63000 (arr. Verviers)           |
| <b>NUTS</b> (Eurostat) | <code>NUTS{0,1,2,3}_202{1,7},</code><br><code>NUTS_MUNICIPALITY_2021</code> | "BE100", "BE211"                                   |
| <b>Postal</b> (bpost)  | <code>POSTAL</code>                                                         | 1000, 2000                                         |
| <b>Interne</b>         | <code>NBB_DISTRICT_2021</code>                                              | "21" (Bxl), "65" (Verviers FR), "66" (Verviers DE) |

Deux dimensions temporelles : - **NIS** : BEFORE\_2019 (591 communes), 2019 (583), 2025 (567) – les communes fusionnent lors des réformes territoriales. - **NUTS** : 2021 (en vigueur) et 2027 (Règlement UE 2026/195).

**Propriétés clés de l'API** : - Codes NIS et POSTAL sont de type **integer** ; codes NUTS et NBB sont de type **character**. - `convert_codes()` renvoie toujours un `data.table(code_from, code_to, nature)`. - Les conversions ambiguës (1:N, M:N) sont bloquées par défaut ; `allow_ambiguous = TRUE` lève cette garde. - `devtools::test()` doit passer (FAIL 0 | WARN 0 | SKIP 8 | PASS 949) avant tout commit.

## 1.3 2. Les 23 classifications supportées

```
get_all_classification_nodes()
```

| Identifiant                               | Systeme | Niveau       | Version     | Type |
|-------------------------------------------|---------|--------------|-------------|------|
| <code>NIS_MUNICIPALITY_BEFORE_2019</code> | NIS     | municipality | BEFORE_2019 | int  |
| <code>NIS_DISTRICT_BEFORE_2019</code>     | NIS     | district     | BEFORE_2019 | int  |
| <code>NIS_PROVINCE_BEFORE_2019</code>     | NIS     | province     | BEFORE_2019 | int  |
| <code>NIS_REGION_BEFORE_2019</code>       | NIS     | region       | BEFORE_2019 | int  |
| <code>NIS_MUNICIPALITY_2019</code>        | NIS     | municipality | 2019        | int  |
| <code>NIS_DISTRICT_2019</code>            | NIS     | district     | 2019        | int  |
| <code>NIS_PROVINCE_2019</code>            | NIS     | province     | 2019        | int  |
| <code>NIS_REGION_2019</code>              | NIS     | region       | 2019        | int  |
| <code>NIS_MUNICIPALITY_2025</code>        | NIS     | municipality | 2025        | int  |
| <code>NIS_DISTRICT_2025</code>            | NIS     | district     | 2025        | int  |
| <code>NIS_PROVINCE_2025</code>            | NIS     | province     | 2025        | int  |
| <code>NIS_REGION_2025</code>              | NIS     | region       | 2025        | int  |
| <code>NIS_COUNTRY</code>                  | NIS     | country      | NA          | int  |
| <code>NUTS_MUNICIPALITY_2021</code>       | NUTS    | municipality | 2021        | chr  |
| <code>NUTS_DISTRICT_2021</code>           | NUTS    | district     | 2021        | chr  |

| Identifiant        | Systeme | Niveau   | Version | Type |
|--------------------|---------|----------|---------|------|
| NUTS_PROVINCE_2021 | NUTS    | province | 2021    | chr  |
| NUTS_REGION_2021   | NUTS    | region   | 2021    | chr  |
| NUTS_COUNTRY       | NUTS    | country  | NA      | chr  |
| NUTS_DISTRICT_2027 | NUTS    | district | 2027    | chr  |
| NUTS_PROVINCE_2027 | NUTS    | province | 2027    | chr  |
| NUTS_REGION_2027   | NUTS    | region   | 2027    | chr  |
| POSTAL             | POSTAL  | postal   | NA      | int  |
| NBB_DISTRICT_2021  | NBB     | district | 2021    | chr  |

**Alias acceptés** : les identifiants tolèrent plusieurs formes abrégées (ex. "CP", "CODE\_POSTAL", "POSTAL" ; "COMMUNE\_2019", "NIS\_COM\_2019"). Voir `normalize_classification_id()`.

**Registre machine** : `CLASSIFICATION_NODES` (liste nommée de 23 entrées, `R/00b_registry.R`) contient pour chaque nœud :

```
system, level, version, code_type, source_table, version_filter,
code_col, label_fr_col, label_nl_col, distinct, detectable, aggregates
```

## 1.4 3. Le graphe de conversion

### 1.4.1 3.1 Aretes primitives

Le graphe est déclaré dans `CONVERSION_GRAPH_EDGES` (`R/00_config.R`). Chaque arête spécifie : `from`, `to`, `relation` (1:1 / N:1 / 1:N / M:N), `notes`, et optionnellement `no_reverse`, `ambiguous_codes`, `coverage`.

Cardinalités :

| Cardinalite | Signification                     | Exemple                                                                            |
|-------------|-----------------------------------|------------------------------------------------------------------------------------|
| 1:1         | bijection exacte                  | NIS_MUNICIPALITY_2019<br>-><br>NUTS_MUNICIPALITY_2021                              |
| N:1         | agregation (N sources -> 1 cible) | NIS_MUNICIPALITY_2019<br>-><br>NUTS_DISTRICT_2021                                  |
| 1:N         | eclatement (1 source -> N cibles) | NIS_MUNICIPALITY_2025<br>-><br>NUTS_DISTRICT_2021<br>pour 3 fusions<br>cross-NUTS3 |
| M:N         | chevauchement complet             | NIS_PROVINCE_2019 -><br>NIS_REGION_2019<br>(Brabant 20000)                         |

### 1.4.2 3.2 Aretes inverses

Chaque arête est automatiquement inversée au chargement du graphe (sauf `no_reverse = TRUE`) avec la cardinalité symétrique (1:1 reste 1:1, N:1 devient 1:N, etc.).

`no_reverse = TRUE` protege les aretes dont l'inversion serait semantiquement incorrecte. Exemple : l'inverse de `NIS_MUNICIPALITY_2025 -> NUTS_DISTRICT_2021 (1:N)` est `1:N` depuis la perspective NUTS3 (beaucoup de communes partageant un NUTS3), pas `N:1`.

### 1.4.3 3.3 Recherche de chemin (BFS)

`find_conversion_path()` (`R/05_conversion_check.R`) fait deux passes : 1. **Passe 1** : BFS sur les aretes `1:1` et `N:1` uniquement – cherche un chemin simple. 2. **Passe 2** : BFS sur toutes les aretes – si aucun chemin simple n'existe.

Si une **arete directe** est declaree entre `from` et `to` avec une cardinalite non-simple, elle prend le dessus sur le chemin indirect trouve par BFS (protection contre les faux simples).

### 1.4.4 3.4 Conversion multi-saut

Les chemins multi-sauts sont **composes** dans `execute_conversion()` : chaque saut intermediaire est execute via `md$crosswalks` ; les resultats sont joints. La colonne `nature` reste `NA` pour les chemins composes (la nature n'est definie que pour les aretes primitives).

### 1.4.5 3.5 Conversions notables

- `NUTS_DISTRICT_2021 <-> NUTS_DISTRICT_2027` : **pas de lien direct**. Trois communes ont change de province entre 2019 et 2025, deplacant leur NUTS3. La conversion doit passer par `NIS` : `NUTS_DISTRICT_2021 -> NIS_COMMUNE -> NIS_MUNICIPALITY_2025 -> NUTS_DISTRICT_2027`.
- `province -> region` est `M:N` (Brabant 20000 couvre 3 regions). Utiliser `NIS_COMMUNE_* -> NIS_REGION_*` (`N:1`) pour un chemin sans ambiguïte.
- **Verviers** (`NIS_DISTRICT_2019 = 63000`) est le seul arrondissement `1:N` vers NUTS3 (BE335 franco-phone + BE336 germanophone).

## 1.5 4. Semantique de perimetre

Chaque arete porte une **semantique de perimetre** calculee par `.edge_perimeter_relation()` et reportee dans `check_conversion_path()` (champ `perimeter_relations`) ainsi que dans `list_available_conversions()` (colonne `perimeter_relation`).

| Valeur          | Definition                                                                                                                                          | Exemples                                                        |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>temporal</b> | Meme systeme, versions differentes. Les limites peuvent evoluer edition par edition mais sans chevauchement entre systemes.                         | <code>NIS_MUNICIPALITY_2019 -&gt; NIS_MUNICIPALITY_2025</code>  |
| <b>identity</b> | Arete <code>1:1</code> – correspondance bijective, meme territoire effectif.                                                                        | <code>NIS_MUNICIPALITY_2019 -&gt; NUTS_MUNICIPALITY_2021</code> |
| <b>nesting</b>  | Arete <code>N:1</code> – <code>N</code> unites fines s'agglomerent en 1 unite grossiere. Le perimetre source est entierement contenu dans la cible. | <code>NIS_MUNICIPALITY_2019 -&gt; NUTS_DISTRICT_2021</code>     |

| Valeur  | Definition                                                                                                                      | Exemples                                |
|---------|---------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| overlap | Arete 1:N ou M:N – une unite source <b>enjambe</b> plusieurs cibles.<br>Seule categorie qui brise la preservation du perimetre. | NIS_DISTRICT_2019 -> NUTS_DISTRICT_2021 |

**Statut global d'un chemin** : - "preserving" : aucun saut n'est overlap. - "crossing" : au moins un saut est overlap.

```
is_perimeter_preserving("NIS_MUNICIPALITY_2019", "NUTS_DISTRICT_2021") # TRUE
is_perimeter_preserving("NIS_MUNICIPALITY_2025", "NUTS_DISTRICT_2021") # FALSE
is_perimeter_preserving("NIS_MUNICIPALITY_2019", "NIS_MUNICIPALITY_2025") # TRUE (temporal)
```

## 1.6 5. La colonne nature

`convert_codes()` renvoie toujours (`code_from`, `code_to`, `nature`). Les valeurs de `nature` :

| Valeur      | Quand                                                                             | Arete              |
|-------------|-----------------------------------------------------------------------------------|--------------------|
| UNCHANGED   | Conversion temporelle<br>NIS, commune<br>inchangee                                | temporal           |
| FUSION      | Conversion temporelle<br>NIS, plusieurs<br>communes fusionnees<br>en une          | temporal           |
| CHANGE_DSTR | Conversion temporelle<br>NIS, commune deplacee<br>dans un autre<br>arrondissement | temporal           |
| CHANGE_PROV | Conversion temporelle<br>NIS, commune deplacee<br>dans une autre province         | temporal           |
| RECODE      | Conversion non<br>temporelle sans<br>enjambement (nesting<br>N:1, identity 1:1)   | nesting / identity |
| OVERLAP     | Conversion non<br>temporelle avec<br>enjambement (1:N ou<br>M:N)                  | overlap            |
| NA          | Chemin multi-saut<br>compose (nature<br>intermediaire non<br>definie)             | multi-saut         |

**Cas BEFORE\_2019 -> 2019** : seules UNCHANGED et FUSION peuvent apparaitre ; 15 communes “orphelines” de BEFORE\_2019 n’ont pas de correspondant en 2019 (`code_to` = NA).

**Cas 2019 -> 2025 – comptes exacts :** - UNCHANGED : 553 communes - FUSION : 27 communes source (13 paires de fusions) - CHANGE\_DSTR : 2 communes (44045 -> 46029, 73040 -> 71072) - CHANGE\_PROV : 1 commune (11056 -> 46030)

**Symetrie temporelle :** la nature est preservee sur le chemin inverse. `convert_codes(46030L, "NIS_MUNICIPALITY_2025", "NIS_MUNICIPALITY_2019", md, allow_ambiguous=TRUE)` renvoie `CHANGE_PROV` pour la ligne `code_to == 11056`.

---

## 1.7 6. API complete

### 1.7.1 6.1 Chargement des donnees

**1.7.1.1 `load_master_data(path = NULL)`** Charge la table de reference pre-construite depuis `inst/extdata/*.rds`. Rapide (pas de fichiers bruts). Retourne une liste nommee avec : `communes`, `postal`, `nis_changes`, `entities`, `crosswalks`.

```
master_data <- load_master_data()
```

**1.7.1.2 `rebuild_master_data(data_dir = NULL, output_dir = NULL)`** Reconstitue le snapshot depuis les fichiers bruts (`data/raw/`). Necessite `readxl`. Sauvegarde dans `inst/extdata/` via `save_master_tables()`.

**1.7.1.3 `load_all_raw_data(data_dir)` / `build_master_table(raw)` / `save_master_tables(md)`**  
Etapas intermediaires du pipeline de construction, exposees pour usage avance.

---

### 1.7.2 6.2 Conversion de codes

**1.7.2.1 `convert_codes(codes, from, to, master_data, allow_ambiguous = FALSE)`** Fonction centrale. Convertit un vecteur de codes.

**Parametres :** - `codes` : vecteur de codes source (integer ou character selon la classification). - `from, to` : identifiants de classification (voir section 2). - `master_data` : objet retourne par `load_master_data()`. - `allow_ambiguous` : si `FALSE` (default), leve `rcl_ambiguous_conversion` pour tout chemin contenant un saut 1:N ou M:N.

**Retour :** `data.table(code_from, code_to, nature)`.

**Comportement pour les codes inconnus :** une ligne avec `code_to = NA` est retournee ; l'avertissement `rcl_unmatched_codes` est emis.

**Erreurs typees :** - `rcl_no_route` : aucun chemin dans le graphe entre `from` et `to`. - `rcl_ambiguous_conversion` : chemin non-simple et `allow_ambiguous = FALSE`.

```
# Simple N:1
convert_codes(c(21004L, 11002L), "NIS_MUNICIPALITY_2019", "NUTS_DISTRICT_2021", master_data)

# Temporel avec nature
convert_codes(c(11002L, 11007L), "NIS_MUNICIPALITY_2019", "NIS_MUNICIPALITY_2025", master_data)
# code_from code_to nature
```

```
#      11002      11002  UNCHANGED
#      11007      11002  FUSION

# Ambigu -- Verviers
convert_codes(63000L, "NIS_DISTRICT_2019", "NUTS_DISTRICT_2021",
              master_data, allow_ambiguous = TRUE)
# code_from code_to nature
#      63000      BE335  OVERLAP
#      63000      BE336  OVERLAP
```

### 1.7.3 6.3 Conversion d'un dataset

**1.7.3.1 convert\_dataset(dt, code\_col, to, master\_data, from = NULL, target\_col = NULL, keep\_code = TRUE, verbose = TRUE, allow\_ambiguous = FALSE, na\_action = "warn")** Convertit une colonne d'un data.table (ou data.frame). Ajoute target\_col (nom auto-généré si NULL). Si from = NULL, auto-détecte la classification source via detect\_classification().

**Paramètres spécifiques :** - target\_col : nom de la colonne résultat. Défaut : "cd\_{to\_lower}". - keep\_code : conserver la colonne source ? Défaut TRUE. - na\_action : "warn" (défaut, garde les NA), "keep" (silencieux), "drop" (supprime les lignes).

```
dt <- data.table(commune = c(21004L, 11002L), pop = c(180000, 530000))
convert_dataset(dt, "commune", "NUTS_DISTRICT_2021", master_data, from = "NIS_MUNICIPALITY_2019")
#      commune      pop cd_nuts3_2021
# 1:   21004   180000      BE100
# 2:   11002   530000      BE211
```

### 1.7.4 6.4 Inspection des chemins

**1.7.4.1 check\_conversion\_path(from, to)** Retourne une liste nommée :

| Champ               | Type              | Contenu                                                               |
|---------------------|-------------------|-----------------------------------------------------------------------|
| is_simple           | logical           | TRUE si tous les sauts sont 1:1 ou N:1                                |
| path                | character         | vecteur des noeuds du chemin                                          |
| relations           | character         | vecteur des cardinalités par saut                                     |
| explanation         | character         | texte lisible                                                         |
| edges_used          | list              | arêtes utilisées (éléments de CONVERSION_GRAPH_EDGES)                 |
| ambiguous_codes     | integer/character | codes ambigus (si applicables)                                        |
| coverage            | character         | couverture (si applicable)                                            |
| perimeter_relations | character         | sémantique par saut : "temporal" / "identity" / "nesting" / "overlap" |
| perimeter_status    | character         | "preserving" ou "crossing"                                            |
| straddle_free       | logical           | TRUE si perimeter_status == "preserving"                              |



**1.7.4.2 is\_perimeter\_preserving(from, to)** Raccourci logique : TRUE si preserving, FALSE si crossing, NA si pas de chemin.

**1.7.4.3 print\_conversion\_check(from, to)** Affiche un resume lisible (simple/ambigu, perimeter status, path, relations). Retourne le resultat `check_conversion_path()` de maniere invisible.

**1.7.4.4 list\_available\_conversions()** Retourne un `data.table` de toutes les aretes du graphe (incluant les inverses) avec : `from`, `to`, `relation`, `perimeter_relation`, `notes`.

```
la <- list_available_conversions()
la[perimeter_relation == "temporal"]      # aretes entre versions NIS
la[perimeter_relation == "overlap"]      # aretes avec enjambement
```

**1.7.4.5 get\_conversion\_matrix()** Retourne un `data.table` (N x N) avec : `from`, `to`, `is_simple`, `relation_chain`. Coûteux ( $O(N^2)$  appels a `check_conversion_path()`).

---

## 1.7.5 6.5 Validation et labels

**1.7.5.1 validate\_codes(codes, classification, master\_data)** Verifie l'appartenance des codes a une classification. Retourne `data.table(code, is_valid)`.

```
validate_codes(c(21004L, 99999L), "NIS_MUNICIPALITY_2019", master_data)
#   code is_valid
# 21004      TRUE
# 99999     FALSE
```

**1.7.5.2 get\_label(codes, classification, master\_data, lang = "fr")** Retourne `data.table(code, label)`. `lang` : "fr" (defaut) ou "nl". Codes inconnus -> `label` = NA.

---

## 1.7.6 6.6 Tables de correspondance

**1.7.6.1 get\_crosswalk(from, to, master\_data, weights = FALSE)** Retourne la table de correspondance complete entre deux classifications. Si `weights` = TRUE, ajoute une colonne `weight` (utilise les poids enregistres via `register_split_weights()`, ou 0.5 egal pour les paires ambiguës sans poids).

```
cw <- get_crosswalk("NIS_MUNICIPALITY_2019", "NUTS_DISTRICT_2021", master_data)
# 583 lignes (une par commune 2019), avec code_from, code_to

get_crosswalk("NIS_DISTRICT_2019", "NUTS_DISTRICT_2021", master_data, weights = TRUE)
# Colonne weight : 0.5/0.5 pour Verviers (63000) par defaut
```

## 1.7.7 6.7 Detection et diagnostic

**1.7.7.1 detect\_classification(codes, master\_data)** Essaie d'identifier automatiquement la classification a partir des valeurs. Retourne la classification la plus probable (string) ou NULL.

**1.7.7.2 diagnose\_classification(dt, col, master\_data, classification = NULL, verbose = TRUE)**

- Si `classification` est fourni : verifie la couverture par rapport a la classification de reference et affiche un tableau de bord.
- Si `classification = NULL` : auto-detection, classe toutes les classifications par taux de correspondance et recommande la meilleure.

**Retour invisible :** liste avec `coverage_rate`, `n_missing`, `n_unknown`, `n_duplicates`, `missing_codes`, `unknown_codes`, `status`.

**status possible :** "COMPLETE", "INCOMPLETE", "COMPLETE\_WITH\_UNKNOWN", "INCOMPLETE\_WITH\_UNKNOWN", "EMPTY", "ALL\_UNKNOWN".

---

## 1.7.8 6.8 Poids et desagregation

**1.7.8.1 split\_ambiguous(dt, code\_col, value\_cols, from, to, master\_data, value\_type = "additive")** Repartit les valeurs d'un `data.table` sur les cibles multiples d'une conversion ambiguë (1:N ou M:N).

- `value_type = "additive"` : les valeurs sont multipliees par le poids.
- `value_type = "ratio"` : les valeurs sont recopiees a l'identique dans chaque cible.

Utilise les poids enregistres (voir ci-dessous), ou 0.5/0.5 egal si aucun poids.

**1.7.8.2 split\_weights\_template(from, to, master\_data)** Retourne un `data.table`(`code_from`, `code_to`, `weight`) pre-rempli avec des poids egaux (1/N). Sert de point de depart pour definir des poids personnalisés.

```
tpl <- split_weights_template("NIS_DISTRICT_2019", "NUTS_DISTRICT_2021", master_data)
# code_from code_to weight
#      63000   BE335    0.5
#      63000   BE336    0.5

tpl[code_from == "63000" & code_to == "BE335", weight := 0.857]
tpl[code_from == "63000" & code_to == "BE336", weight := 0.143]
```

**1.7.8.3 register\_split\_weights(from, to, weights\_dt, variable = "default")** Enregistre des poids personnalisés pour la session (environnement interne, pas persistant). `variable` permet de distinguer plusieurs jeux de poids (ex. "population", "emploi").

**1.7.8.4 get\_split\_weights(from, to, variable = "default")** Recupere les poids enregistres. Leve `rcl_no_weights` si aucun poids n'est enregistre.

**1.7.8.5 list\_split\_weights()** Retourne un `data.table` listant tous les jeux de poids enregistrés dans la session.

**1.7.8.6 clear\_split\_weights()** Efface tous les poids enregistrés dans la session.

---

## 1.7.9 6.9 Rebasage longitudinal

**1.7.9.1 rebase\_series(dt, period\_col, code\_col, value\_cols, version\_map, to, master\_data, fun = sum, value\_type = "additive", split = "default")** Convertit un panel longitudinal spanning plusieurs versions NIS vers une version cible.

**Parametres :** - `period_col` : colonne des périodes (année, trimestre, etc.). - `code_col` : colonne des codes géographiques. - `value_cols` : colonnes de valeurs à convertir. - `version_map` : `list("NIS_MUNICIPALITY_2019" = 2022L, "NIS_MUNICIPALITY_2025" = 2025L)`. Chaque entrée associe une classification à un ou plusieurs périodes. - `to` : classification cible. - `fun` : fonction d'aggrégation pour `value_type = "additive"` (défaut `sum`). - `value_type` : "additive" (ex. effectifs) ou "ratio" (ex. taux). - `split` : "default" (utilise les poids enregistrés) ou objet `data.table` de poids.

**Avertissements émis :** - `rcl_missing_periods` : certaines périodes du dataset ne sont pas couvertes par `version_map`.

```
panel <- data.table(
  year = c(2022L, 2022L, 2025L),
  commune = c(11002L, 11007L, 11002L),
  pop = c(530000, 42000, 590000)
)

rebase_series(
  panel,
  period_col = "year",
  code_col = "commune",
  value_cols = "pop",
  version_map = list("NIS_MUNICIPALITY_2019" = 2022L, "NIS_MUNICIPALITY_2025" = 2025L),
  to = "NIS_MUNICIPALITY_2025",
  master_data = master_data
)
# 2022 : commune 11002 -> 530000 + 42000 = 572000 (fusionne)
# 2025 : 590000 (inchange)
```

---

## 1.7.10 6.10 Correspondance floue

**1.7.10.1 fuzzy\_match\_names(names, target, master\_data, max\_dist = 0.3, language = "both")** Rattache des noms (orthographe approchée) à des codes officiels. Nécessite le package `stringdist` (Suggests).

**Retour :** `data.table(input_name, matched_name, matched_code, distance, is_confident)`.

```
fuzzy_match_names(
  c("Bruxelles", "Antwerpen", "Liege"),
  target = "NIS_MUNICIPALITY_2019",
  master_data,
  max_dist = 0.3
)
```

**1.7.10.2 identify\_from\_names(names, master\_data, ...)** Variante de `fuzzy_match_names` avec auto-detection de la classification cible.

---

## 1.7.11 6.11 Objet nomenclature

Interface programmatique pour manipuler les classifications dynamiquement.

**1.7.11.1 nomenclature(system, level, version = NA)** Construit un objet `nomenclature` valide contre `CLASSIFICATION_NODES`.

```
n <- nomenclature("NIS", "municipality", "2019")
nom_system(n)    # "NIS"
nom_level(n)     # "municipality"
nom_version(n)   # "2019"
```

**1.7.11.2 list\_nomenclatures() / nomenclature\_levels(system) / nomenclature\_versions(system, level)** Introspection de l'espace des classifications valides.

**1.7.11.3 nomenclature\_children(n) / nomenclature\_parents(n)** Navigation dans la hierarchie d'agregation definie par le champ `aggregates` de `CLASSIFICATION_NODES`.

**1.7.11.4 is\_nomenclature(x)** Test de type.

---

## 1.7.12 6.12 Visualisation

Ces fonctions necessitent `visNetwork` (Suggests).

**1.7.12.1 visualize\_classification\_graph(master\_data = NULL)** Graphe interactif des relations entre classifications (noeuds = classifications, aretes = conversions avec cardinalite).

**1.7.12.2 visualize\_conversion\_matrix(master\_data = NULL)** Matrice de faisabilite des conversions (simple / ambigu / impossible).

**1.7.12.3 visualize\_hierarchy(version, master\_data, max\_communes = 20)** Arbre hiérarchique NIS pour une version donnée ("NIS\_2019" ou "NIS\_2025").

---

## 1.8 7. Architecture interne

```
R/
00_config.R          -- CLASSIFICATION_REGISTRY, CONVERSION_GRAPH_EDGES, constantes NIS
00b_registry.R       -- CLASSIFICATION_NODES (source de verité unique, 23 noeuds)
00c_nomenclature.R   -- objet nomenclature + S3 methods
01_load_data.R       -- chargement et parsing des fichiers bruts (Excel)
02_build_master_table.R -- construction de entities + crosswalks + tables sources
03_convert.R         -- convert_codes(), execute_conversion(), route_conversion()
04_fuzzy_match.R     -- fuzzy_match_names(), identify_from_names()
05_conversion_check.R -- check_conversion_path(), BFS, build_conversion_graph()
06_visualize.R       -- visualize_*()
07_dataset_convert.R -- convert_dataset()
07_detect.R          -- detect_classification()
07_diagnose.R        -- diagnose_classification()
07_split_ambiguous.R -- split_ambiguous(), split_weights_template(), registre de poids
08_load_prebuilt.R   -- load_master_data(), save_master_tables()
09_query.R           -- validate_codes(), get_label(), get_crosswalk(), .node_reference_codes()
10_rebase.R          -- rebase_series()
classifications.R    -- documentation de classification_reference (?classification_reference)
data.R               -- documentation des datasets internes
zzz.R                -- utils::globalVariables() (supprimer les NOTES R CMD CHECK)
```

### 1.8.1 7.1 Flot d'une conversion

```
convert_codes(codes, from, to, md)
|
+-- check_conversion_path(from, to)    -- valide et trouve le chemin (cache BFS)
|   build_conversion_graph()          -- construit l'adjacence depuis CONVERSION_GRAPH_EDGES
|   find_conversion_path()            -- passe 1 (simple) puis passe 2 (tous)
|
+-- execute_conversion(codes, from, to, md)
    route_conversion(from, to, md)    -- lecture de md$crosswalks pour le saut direct
    .crosswalk_hop(from, to, md)      -- lookup data.table [from_id == F & to_id == T]
    .compose_via_crosswalks(path, md) -- composition pour les chemins multi-sauts
    .normalize_conversion_result()     -- force le schema (code_from, code_to, nature)
                                         -- remplit nature = RECODE / OVERLAP selon perimetre
```

### 1.8.2 7.2 CLASSIFICATION\_NODES

Chaque entree est une liste avec :

```
CLASSIFICATION_NODES[["NIS_MUNICIPALITY_2019"]]
# $system      "NIS"
# $level       "municipality"
# $version     "2019"
```

```
# $code_type      "integer"
# $source_table   "communes"
# $version_filter "2019"
# $code_col       "cd_commune"
# $label_fr_col   "tx_commune_fr"
# $label_nl_col   "tx_commune_nl"
# $distinct       FALSE
# $detectable     TRUE
# $aggregates     character(0)  # commune est un noeud feuille
```

Le champ `aggregates` forme un DAG (pas un arbre) : NIS\_DISTRICT\_2019 est agrege par la province ET par la region ; NUTS\_COUNTRY agrege NUTS\_REGION\_2021 ET NUTS\_REGION\_2027.

### 1.8.3 7.3 Moteur de conversion (Phase 2)

L'executeur lit **exclusivement** `md$crosswalks` pour chaque saut individuel. Les anciens handlers fermes ont ete supprimes. Le test `test-route-parity.R` garantit que toute conversion declaree simple est executee correctement et que les crosswalks reproduisent exactement les resultats du moteur (fixture golden + parite par arete).

### 1.8.4 7.4 Normalisation du resultat

`.normalize_conversion_result(dt, from, to)` remplit la colonne `nature` : - Chemins temporels (meme systeme, versions differentes) : `nature` deja portee par `md$crosswalks` (UNCHANGED/FUSION/CHANGE\_DSTR/CHANGE\_PROV). - Chemins non-temporels : `nature` = "RECODE" si perimetre `identity/nesting`, "OVERLAP" si perimetre `overlap`. - Chemins multi-sauts (appel sans `from/to`) : `nature` reste NA.

---

## 1.9 8. Modele de donnees master\_data

`load_master_data()` retourne une liste avec cinq tables :

### 1.9.1 communes – table large

Une ligne par (`cd_commune`, `nis_version`). Colonnes principales :

|                            |                  |                                                 |
|----------------------------|------------------|-------------------------------------------------|
| <code>cd_commune</code>    | <code>int</code> | -- code NIS de la commune                       |
| <code>nis_version</code>   | <code>chr</code> | -- "BEFORE_2019"   "2019"   "2025"              |
| <code>tx_commune_fr</code> | <code>chr</code> | -- nom francais                                 |
| <code>tx_commune_nl</code> | <code>chr</code> | -- nom neerlandais                              |
| <code>cd_arr</code>        | <code>int</code> | -- code arrondissement NIS                      |
| <code>cd_prov</code>       | <code>int</code> | -- code province NIS                            |
| <code>cd_region</code>     | <code>int</code> | -- code region NIS                              |
| <code>cd_nuts_lau</code>   | <code>chr</code> | -- code NUTS LAU 2021                           |
| <code>cd_nuts3</code>      | <code>chr</code> | -- code NUTS3 2021                              |
| <code>cd_nuts2</code>      | <code>chr</code> | -- code NUTS2 2021                              |
| <code>cd_nuts1</code>      | <code>chr</code> | -- code NUTS1 2021                              |
| <code>cd_nuts3_2027</code> | <code>chr</code> | -- code NUTS3 2027 (NULL si pas encore calcule) |
| <code>cd_internal</code>   | <code>chr</code> | -- code NBB_DISTRICT_2021 (2 chiffres)          |

### 1.9.2 postal – codes postaux

```
cd_postal  int  -- code postal (4 chiffres)
cd_commune int  -- commune NIS 2019 correspondante
```

### 1.9.3 nis\_changes – transitions de version

```
from_version chr -- "BEFORE_2019" ou "2019"
to_version   chr -- "2019" ou "2025"
code_from    int -- code source
code_to      int -- code cible
nature       chr -- "UNCHANGED" | "FUSION" | "CHANGE_DSTR" | "CHANGE_PROV"
```

### 1.9.4 entities – enumeration des membres par classification

```
classification_id chr -- identifiant de la classification
code              chr -- code (toujours character ici, coerce si besoin)
name_fr          chr -- nom francais
name_nl          chr -- nom neerlandais
```

Une ligne par (classification\_id, code). Sert a validate\_codes(), get\_label(), detect\_classification().

### 1.9.5 crosswalks – liens primitifs entre classifications

```
from_id  chr -- classification source
to_id    chr -- classification cible
code_from chr -- code source
code_to  chr -- code cible (NA si code source sans correspondant)
relation chr -- "1:1" | "N:1" | "1:N" | "M:N"
nature   chr -- nature de la conversion (voir section 5)
```

Une ligne par lien primitif. C'est la table lue par route\_conversion() pour chaque saut du chemin. La fixture golden (tests/testthat/fixtures/golden\_crosswalks.rds) fige son contenu ; tout changement dans les crosswalks impose de regenerer la fixture avec gen\_golden().

---

## 1.10 9. Classes d'erreur et avertissements

Toutes les erreurs et avertissements utilisant rlang::abort() / rlang::warn() portent une classe rcl\_\*. Interceptables avec tryCatch(..., rcl\_xxx = handler).

| Classe                   | Type    | Declencheur                                                    |
|--------------------------|---------|----------------------------------------------------------------|
| rcl_no_route             | erreur  | convert_codes() : aucun chemin de from vers to                 |
| rcl_ambiguous_conversion | erreur  | convert_codes() : chemin non-simple et allow_ambiguous = FALSE |
| rcl_unmatched_codes      | warning | codes inconnus dans la source, code_to = NA                    |

| Classe              | Type    | Declencheur                                                |
|---------------------|---------|------------------------------------------------------------|
| rcl_missing_periods | warning | rebase_series() : periodes non couvertes par version_map   |
| rcl_no_weights      | erreur  | get_split_weights() : aucun poids enregistre pour la paire |
| rcl_invalid_weights | erreur  | poids qui ne somment pas a 1, ou codes absents             |
| rcl_schema_error    | erreur  | table communes non conforme au schema attendu au build     |

## 1.11 10. Suite de tests

tests/testthat/

```

  helper-master_data.R      -- charge master_data une fois (session-level fixture)
  helper-golden.R           -- gen_golden() : regenere la fixture golden

  test-conversions.R        -- cas principaux convert_codes() (NIS, NUTS, POSTAL)
  test-conversion-check.R   -- check_conversion_path(), is_perimeter_preserving()
  test-perimeter-semantics.R -- .edge_perimeter_relation(), straddle_free
  test-temporal-nature.R    -- CHANGE_DSTR / CHANGE_PROV / UNCHANGED / FUSION
  test-crosswalks-golden.R  -- golden fixture : sorties figees du moteur
  test-crosswalks-parity.R  -- parite crosswalk <-> moteur par arete
  test-route-parity.R       -- toute conversion simple est executable
  test-registry.R           -- CLASSIFICATION_NODES : structure, count (23)
  test-registry-consistency.R -- coherence CLASSIFICATION_NODES <-> CLASSIFICATION_REGISTRY
  test-split-registry.R     -- registre de poids (register/get/clear)
  test-convert-dataset.R    -- convert_dataset()
  test-rebase.R             -- rebase_series()
  test-diagnose.R           -- diagnose_classification()
  test-query.R              -- validate_codes(), get_label(), get_crosswalk()
  test-misc.R               -- alias, normalize_classification_id(), edge cases
  test-edge-cases.R         -- codes NA, vecteurs vides, from == to
  test-nomenclature.R       -- objet nomenclature, S3 methods, hierarchie
  test-fuzzy.R              -- fuzzy_match_names() (skip si stringdist absent)
  test-load.R               -- load_master_data(), structure des tables

```

**Etat cible :** FAIL 0 | WARN 0 | SKIP 8 | PASS 949. Les 8 SKIPS sont des tests fuzzy\_match\_names gardes par skip\_if\_not\_installed("stringdist").

**Regle golden :** toute modification qui change le contenu des crosswalks ou les valeurs de nature impose de regenerer la fixture :

```

devtools::load_all(".")
source("tests/testthat/helper-golden.R")
gen_golden() # ecrit tests/testthat/fixtures/golden_crosswalks.rds
# puis git add + commit la fixture

```

Ne **jamais** supprimer golden\_crosswalks.rds – regenerer uniquement.



## 1.12 11. Reconstruction du snapshot

Le package fonctionne normalement sur le snapshot pre-construit (`inst/extdata/*.rds`). La reconstruction est necessaire uniquement apres modification des fichiers bruts.

### 1.12.1 Fichiers bruts requis (`data/raw/`)

| Fichier                          | Contenu                                                              |
|----------------------------------|----------------------------------------------------------------------|
| CONVERSION_NIS2019_NUTS2021.xlsx | NIS 2019 vers NUTS 2021 (Statbel)                                    |
| CONVERSION_NIS2025_NUTS2027.xlsx | NIS 2025 vers NUTS 2027 (Statbel)                                    |
| NIS_CHANGES.xlsx                 | Transitions de version NIS (BEFORE_2019 -> 2019 -> 2025) avec nature |
| POSTAL_NIS.xlsx                  | Codes postaux -> communes NIS                                        |
| NUTS2021_LABELS.xlsx             | Libelles NUTS 2021 FR/NL                                             |
| NUTS2027_LABELS.xlsx             | Libelles NUTS 2027 FR/NL                                             |

### 1.12.2 Commandes

```
# Reconstruction complete
master_data <- rebuild_master_data()      # charge les bruts, construit, sauvegarde

# Ou etapes separees
raw  <- load_all_raw_data("data/raw/")
md   <- build_master_table(raw)
save_master_tables(md)                   # ecrit inst/extdata/*.rds
```

### 1.12.3 Schema de validation

`build_master_table()` valide le schema de la table `communes` au build et leve `rcl_schema_error` si des colonnes obligatoires sont absentes ou si les types ne correspondent pas au registre `CLASSIFICATION_NODES`.

## 1.13 12. Datasets d'exemple

Le package inclut huit `data.table` prêts a l'emploi (charges avec `data()`). Toutes les valeurs socio-economiques sont **fictives** (generees par `set.seed(42)`).

### 1.13.1 Datasets propres (couverture complete)

Chaque dataset contient une ligne par unite geographique, avec cinq colonnes : la colonne `code` + `population`, `emplois`, `masse_sal`, `taux_activite`.

| Dataset                                  | Code col               | N lignes | Classification        |
|------------------------------------------|------------------------|----------|-----------------------|
| <code>rc_full_municipalities_2019</code> | <code>desc_2019</code> | 583      | NIS_MUNICIPALITY_2019 |
| <code>rc_full_municipalities_2025</code> | <code>desc_2025</code> | 567      | NIS_MUNICIPALITY_2025 |

| Dataset                | Code col      | N lignes | Classification     |
|------------------------|---------------|----------|--------------------|
| rc_full_districts_2019 | cd_arr        | 44       | NIS_DISTRICT_2019  |
| rc_full_regions_2019   | cd_region     | 3        | NIS_REGION_2019    |
| rc_full_nuts3_2021     | cd_nuts3      | 44       | NUTS_DISTRICT_2021 |
| rc_full_nuts3_2027     | cd_nuts3_2027 | 44       | NUTS_DISTRICT_2027 |
| rc_full_postal         | cd_postal     | 1 149    | POSTAL             |

### 1.13.2 Dataset sale (rc\_dirty\_municipalities\_2019)

319 lignes basees sur NIS\_MUNICIPALITY\_2019 avec quatre types d'anomalies intentionnelles, pour demon-  
strer `diagnose_classification()` et `validate_codes()` :

| Anomalie                  | Quantite                         |
|---------------------------|----------------------------------|
| Doublons de lignes        | 5 communes repetees              |
| Codes d'une autre version | 4 codes NIS 2025 absents de 2019 |
| Code NA                   | 2 lignes                         |
| Valeur population NA      | 8 lignes                         |

```
data(rc_dirty_municipalities_2019)
master_data <- load_master_data()

# Diagnostic complet
diagnose_classification(rc_dirty_municipalities_2019, "cd_commune", master_data,
                        classification = "NIS_MUNICIPALITY_2019")

# Validation code par code
validate_codes(rc_dirty_municipalities_2019$cd_commune,
               "NIS_MUNICIPALITY_2019", master_data)

# Auto-detection (si la classification n'est pas connue a l'avance)
diagnose_classification(rc_dirty_municipalities_2019, "cd_commune", master_data)
```

### 1.13.3 Regeneration

Les fichiers `.rda` dans `data/` sont produits par :

```
source("data-raw/generate_example_datasets.R")
```

La graine aleatoire `set.seed(42)` garantit la reproductibilite.

## 1.14 Glossaire

| Terme                          | Definition                                                                                              |
|--------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>NIS</b>                     | Nomenclature INS/NIS de Statbel (Institut National de Statistique belge)                                |
| <b>NUTS</b>                    | Nomenclature des Unites Territoriales Statistiques d'Eurostat                                           |
| <b>LAU</b>                     | Local Administrative Unit (= commune belge dans la nomenclature NUTS)                                   |
| <b>crosswalk</b>               | Table de correspondance entre deux classifications (une ligne par lien primitif)                        |
| <b>nature</b>                  | Qualification d'une conversion : UNCHANGED / FUSION / CHANGE_DSTR / CHANGE_PROV / RECODE / OVERLAP / NA |
| <b>perimetre</b>               | Territoire physique delimite sur la carte (distinct du code qui l'etiquette)                            |
| <b>semantique de perimetre</b> | Classification d'un saut selon son impact perimetre : temporal / identity / nesting / overlap           |
| <b>straddle</b>                | Un perimetre source qui enjambe deux ou plusieurs perimetres cibles (arete <b>overlap</b> )             |
| <b>Verviers</b>                | Arrondissement NIS 63000 – seul cas 1:N vers NUTS3 (BE335 FR + BE336 DE)                                |
| <b>BFS</b>                     | Breadth-First Search – algorithme de recherche de chemin dans le graphe de conversion                   |
| <b>fixture golden</b>          | Snapshot des sorties du moteur, verifie a chaque run pour detecter toute regression                     |